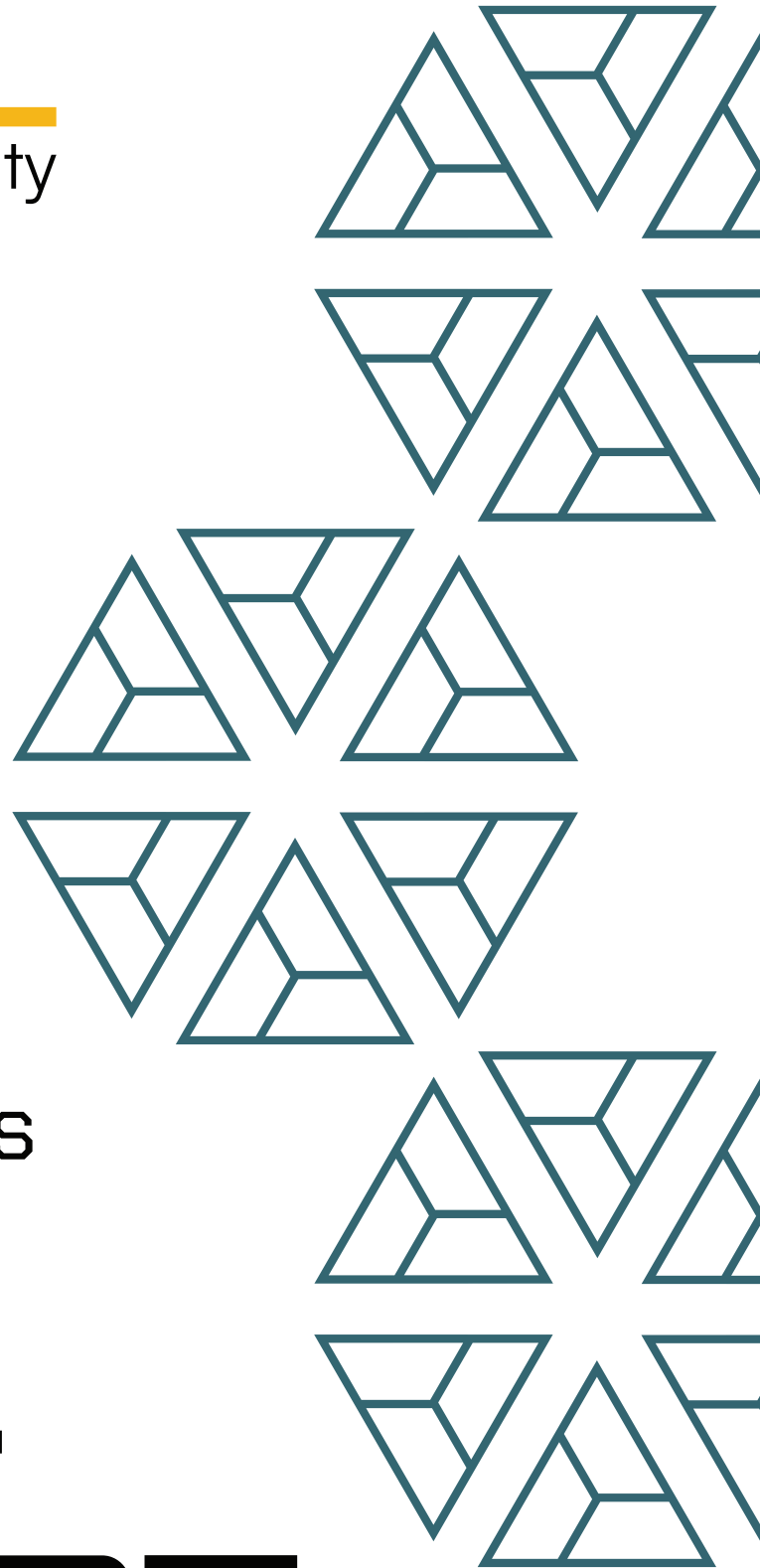




BAIL
security



Golem
Core / Strategies

FINAL REPORT

May '2026

Disclaimer:

Security assessment projects are time-boxed and often reliant on information that may be provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

The content of this assessment is not an investment. The information provided in this report is for general informational purposes only and is not intended as investment, legal, financial, regulatory, or tax advice. The report is based on a limited review of the materials and documentation provided at the time of the audit, and the audit results may not be complete or identify all possible vulnerabilities or issues. The audit is provided on an "as-is," "where-is," and "as-available" basis, and the use of blockchain technology is subject to unknown risks and flaws.

The audit does not constitute an endorsement of any particular project or team, and we make no warranties, expressed or implied, regarding the accuracy, reliability, completeness, or availability of the report, its content, or any associated services or products. We disclaim all warranties, including the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

We assume no responsibility for any product or service advertised or offered by a third party through the report, any open-source or third-party software, code, libraries, materials, or information linked to, called by, referenced by, or accessible through the report, its content, and the related services and products. We will not be liable for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract.

The contract owner is responsible for making their own decisions based on the audit report and should seek additional professional advice if needed. The audit firm or individual assumes no liability for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract. The contract owner agrees to indemnify and hold harmless the audit firm or individual from any and all claims, damages, expenses, or liabilities arising from the use or reliance on the audit report or the smart contract.

By engaging in a smart contract audit, the contract owner acknowledges and agrees to the terms of this disclaimer.

1. Project Details

Important:

Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Golem – Core / Strategies - Audit Report
Website	golem.foundation
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/golemfoundation/octant-v2-core/tree/f5ca638e43fd330df5c4b783d3e5c166902a5a0a
Resolution 1	https://github.com/golemfoundation/octant-v2-core/tree/b2833c7aed7449d604319c6f99feb281a8219caa

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no changes made)	Failed resolution	Open
High						
Medium	10	5		5		
Low	38	16	2	20		
Informational	26	16		10		
Governance	4	1		3		
Total	78	38	2	38		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

TokenizedStrategy

`TokenizedStrategy` is the core abstract implementation contract for all Octant V2 strategies, forked from Yearn's `TokenizedStrategy` and modified to support yield donation functionality, enhanced security, and role-based access control. It serves as the `delegatecall` target for all strategies inheriting `BaseStrategy`, handling ERC-4626 vault accounting, ERC-20 share token management, EIP-2612 permit support, and the two-step dragon router change mechanism with a mandatory 14-day cooldown period. Two specialized implementations extend it: `YieldDonatingTokenizedStrategy` and `YieldSkimmingTokenizedStrategy`.

The contract manages the full `ERC-4626` deposit/withdraw lifecycle, tracking `totalAssets` manually to prevent price-per-share manipulation via direct token transfers. Profits are credited to the dragon router through specialized `report` implementations in child contracts rather than using a gradual unlock mechanism.

Privileged Functions

- `tend`
- `shutdownStrategy`
- `emergencyWithdraw`
- `setPendingManagement`
- `acceptManagement`
- `setKeeper`
- `setEmergencyAdmin`
- `setDragonRouter`
- `cancelDragonRouterChange`
- `setName`
- `setEnableBurning`

Core Invariants:

INV 1: Dragon router changes require a mandatory 14-day cooldown period before finalization.

INV 2: Once a strategy is shut down, it cannot be reactivated, and no new deposits or mints are accepted.

INV 3: Reentrancy is prevented on all state-changing deposit, withdrawal, and reporting functions via the `nonReentrant` modifier.

Issue_01	Losses can be bypassed by withdrawing before reports
Severity	Medium
Description	If a loss is realized in the underlying vault, but the same loss is not yet communicated to this strategy contract via a report call, the share mechanism can completely break down. In the strategy, when shares are redeemed, the contract calls <code>withdraw</code> on the vault after calculating the amount of assets owed to it via <code>convertToAssets</code>. If the loss isn't reported here yet, it will use a stale PPS (say 1) to try to withdraw 100 assets by burning 100 shares from the user, even though the vault actually realized a 10% loss. The main reason is that the funds are withdrawn from the vault using <code>withdraw(asset)</code> instead of <code>redeem(shares)</code>. So the incorrect assets calculated using the stale ratio are withdrawn, redeeming the user's shares 1:1. This, however, creates a big issue for the rest of the users, since the strategy now has a higher loss, since it allowed 1:1 redemption for a user. Say the vault originally had 1000 assets, which, after a loss, went down to 900. Since the user was given 100 tokens for 100 shares, there are now actually only 800 tokens for 900 shares, resulting in a loss of ~12%, higher than the original 10%. This issue is of higher severity for the Yearn vaults, since it does not use gradual interest accrual like aave/spark vaults, and instead depends on <code>process_report</code> calls on the underlying vault. For those strategies, it is easy to trigger this scenario by looking for a loss realizing <code>process_report</code> call and then immediately withdrawing from the strategy, before the corresponding report call is made, since these 2 report calls will be made by different entities. Thus, any user monitoring the chain can bypass the loss realization even though it is confirmed in the underlying vault. This does not even require frontrunning and is thus easy to execute.
Recommendations	Consider implementing <code>freeFunds</code> in the strategies in terms of

	shares, using redeem. This way, freeFunds can actually return fewer assets than requested in case of a loss, which is the way TokenizedStrategy expects it to work. Another option would be to implement a delayed withdrawal mechanism.
Comments / Resolution	Acknowledged. Withdraw frontruns are inherent to erc4626 vaults, and off-chain monitoring will be set up to prevent such scenarios.

Issue_02	Missing slippage protection
Severity	Medium
Description	The <code>deposit/mint</code> and <code>withdraw/redeem</code> functions all operate on the stored <code>totalAssets</code> and <code>totalShares</code> values, and the share ratio can change between transaction submission and execution. This can lead to unexpected losses during withdrawals. For example, say a user controls 1000 shares, which represent 1000 tokens. Now, assume the underlying strategy suffers a loss, so the underlying now only holds 990 tokens. If this loss is already reported via report, the <code>totalAssets</code> reflect this new value, and a full redemption of the user's position will only net them 990 tokens instead of the full 1000 while burning all 1000 shares. The redeem function does have a <code>maxLoss</code> parameter, but this does not always come into play. If the underlying yearn vault has enough idle funds, it doesn't even apply <code>max_loss</code>, and completely bypasses it. <pre>if requested_assets > current_total_idle: # logic bypassed due to idle funds available if assets > requested_assets and max_loss < MAX_BPS: # above check fails, since assets=requested_assets</pre> So even though the strategy incurred a loss, which was reported by yearn V3 via its <code>_process_report</code> function, which updated its <code>current_debt</code>, and even though that loss got reported to golem via the report function updating its <code>totalAssets</code>, the <code>maxLoss</code> parameter never comes into play since there were enough idle assets. Thus, users could be expecting to receive 1000 tokens but instead receive 990 tokens, even if they pass 0% max loss, making this unexpected.
Recommendations	Consider adding a <code>minimumOut</code> parameter as well.

Comments / Resolution	Acknowledged.
-----------------------	---------------

Issue_03	Permit frontrun grief
Severity	Low
Description	The <code>permit</code> function on <code>TokenizedStrategy</code> is permissionless, like the standard <code>ERC20</code> permit. A malicious user can execute a transaction using the signed calldata from the mempool, frontrunning the actual user's own transaction, which increments the owner's nonce, and sets allowance per the signature. This will then fail the actual owner's transaction. The owner pays for the failed gas and must produce a fresh signature.
Recommendations	Consider acknowledging this behaviour.
Comments / Resolution	Acknowledged and documented.

Issue_04	Dragon router change strands profit shares
Severity	Low
Description	<code>TokenizedStrategy finalizeDragonRouterChange</code> only stores the new <code>dragonRouter</code> and clears the pending state. It does not move strategy share balances from the former router to the successor. <code>YieldDonatingTokenizedStrategy</code> mints profit shares to whatever <code>dragonRouter</code> is active on each report and runs loss protection burns only against the current <code>dragonRouter</code> balance. After finalization, the old router keeps any shares it already received and behaves like any other shareholder, including redeeming into the underlying without going through the forwarder pipeline that the system expects. This causes a stepwise drop in the loss protection across users. When burning is enabled, a subsequent loss applies burns against the new router only. That address may hold far fewer or no shares compared to the previous dragon router. Due to the old router still holding the prior donation shares. Because of this, users will have to bear the loss despite the existing shares that were intended to protect the users from this scenario.
Recommendations	During <code>finalizeDragonRouterChange</code> , consider moving all strategy shares from the old dragon router to the new address.
Comments / Resolution	Acknowledged.

Issue_05	<code>emergencyWithdraw</code> can revert since the amount available for withdraw can change in between
Severity	Low
Description	The <code>emergencyWithdraw</code> function attempts to withdraw the input amount from the underlying strategy for most of the implementations. In case some user queues withdrawal in between, if the available funds of the strategy drop below the input amount, the <code>emergencyWithdraw</code> function will revert.
Recommendations	Have a wildcard value to withdraw all the funds that are currently available in case of an emergency.
Comments / Resolution	Acknowledged.

Issue_06	<code>maxDeposit</code> will be incorrect in case <code>totalAssets == 0 && totalSupply > 0</code>
Severity	Low
Description	The value returned by <code>maxDeposit</code> , i.e., <code>availableDepositLimit</code> will be incorrect as the deposit function will revert in case <code>totalAssets == 0 && totalSupply > 0</code> since the shares calculated will be 0
Recommendations	Return 0 in case <code>totalAssets == 0 && totalSupply > 0</code>
Comments / Resolution	Fixed following recommendation.

Issue_07	Shutdown does not revoke unlimited token approvals
Severity	Informational
Description	Shutdown is permanent and blocks all future deposits and mints, but the strategies keep their unlimited token approvals to external protocols. Those approvals are no longer needed once the vault is shut down. If the integrated protocol is compromised or behaves maliciously, any assets later sitting idle in the strategy can still be pulled through the standing approval. This includes funds recovered through emergency withdrawal or any later token transfers to the strategy. As a result, the shutdown does not fully isolate the vault from the external integration it is trying to exit.
Recommendations	Consider revoking approvals on shutdown.
Comments / Resolution	Fixed, approvals are now granted on every transaction as needed.

Issue_08	Emergency withdraw omits on-chain event
Severity	Informational
Description	The <code>emergencyWithdraw</code> function in <code>TokenizedStrategy</code> runs only after <code>shutdownStrategy</code> sets the shutdown flag. It forwards to <code>shutdownWithdraw</code> on the strategy, which calls <code>_emergencyWithdraw</code> on concrete strategies to pull asset from yield into idle balance. <code>shutdownStrategy</code> emits <code>StrategyShutdown</code>, so indexers record shutdown, but the <code>emergencyWithdraw</code> function itself does not emit any event indicating the shutdown being processed.
Recommendations	Consider emitting an event in the <code>emergencyWithdraw</code> function as well.
Comments / Resolution	Fixed following recommendations.

Issue_09	Comment on <code>lastReport</code> says block, not timestamp
Severity	Informational
Description	In <code>TokenizedStrategy</code> the inline comment immediately above the <code>S.lastReport</code> assignment in the initialize function states that the field records this block. The assignment, however, stores uint96 of the execution timestamp in seconds, not the block height. Thus, the comment is incorrect.
Recommendations	Fix the comment indicating that it is the block timestamp, not the block number.
Comments / Resolution	Fixed following recommendations.

Issue_10	0 recovered address not rejected
Severity	Informational
Description	In the <code>permit</code> function, the contract extracts the signer from the passed-in signature parameters. <pre>[address recoveredAddress, ,] = ECDSA.tryRecover(digest, v, r, s); if (recoveredAddress != owner) { revert TokenizedStrategy__InvalidSigner(); }</pre> The contract uses <code>ECDSA.tryRecover</code> to get the signer, which can return 0 in case of an invalid signer. The contract, however, does not revert on a 0 result or reject this value. This result should be explicitly rejected since it is invalid.
Recommendations	Revert in case the <code>recoveredAddress</code> is 0.
Comments / Resolution	Acknowledged.

Issue_11	Deposit limit violated via donations
Severity	Informational
Description	The strategy contract implements a deposit limit via the <code>_maxDeposit</code> function, disallowing deposits above this limit into the contract or into their underlying strategies. This limit is, however, bypassed if any user directly donates tokens to the strategy. The strategy uses any token balance in its own contract as an idle balance, fully participating in the system.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_12	Fee-on-transfer tokens not supported
Severity	Informational
Description	The deposit function transfers the tokens to itself and does not use a pre-post balance check to record the actual amount of tokens transferred. Thus, this contract is not compatible with any tokens that charge a transfer fee, since that would require pre- and post-balance checks.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

BaseStrategy

BaseStrategy is an abstract contract that provides the foundational framework for building permissionless ERC-4626 compliant vaults within the Octant V2 system. It utilizes an immutable proxy pattern where all standard logic, accounting, and **ERC-4626** functionality are delegated to a separate **TokenizedStrategy** implementation contract via delegatecall, allowing strategists to focus solely on implementing yield-generation logic. It is inherited by all strategy contracts in the system.

The contract operates through a delegatecall fallback mechanism that forwards any undefined function calls to the **TokenizedStrategy** implementation at a fixed address stored at the **EIP-1967** proxy slot. Strategists must override three core functions—**_deployFunds**, **_freeFunds**, and **_harvestAndReport**—to define how assets are deployed into yield sources, freed for withdrawals, and accounted for during reports.

Core Invariants:

INV 1: Only delegatecall context (`msg.sender == address(this)`) can invoke the strategy callback hooks **deployFunds**, **freeFunds**, **harvestAndReport**, **tendThis**, and **shutdownWithdraw**.

Issue_13	Tend functionality not implemented
Severity	Informational
Description	None of the strategies utilizes the tend functionality or implements the _tend function. Consider removing this functionality if not required.
Recommendations	Consider removing the tend functionality if not required, since none of the strategies available utilize it.
Comments / Resolution	Acknowledged.

Issue_14	Raw eth not supported
Severity	Informational
Description	The base strategy contract does not implement a receive function or have payable functions in the strategies. Thus, raw ETH/ gas tokens are not supported by the protocol at all.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Privileged

Privileged is an abstract contract that provides privileged address storage and management functions for strategy contracts requiring whitelist-gated access control. It uses **ERC-7201** namespaced storage (**PRIVILEGED_STORAGE** slot) to avoid storage collisions in the delegatecall proxy pattern used by the strategy system. It is inherited by **PrivilegedYieldDonatingTokenizedStrategy** and **PrivilegedYieldSkimmingTokenizedStrategy** to gate deposit and mint operations.

The contract maintains a mapping of addresses to a boolean privileged status, with internal setter functions [**_setPrivileged** and **_setPrivilegedBatch**] for individual and batch updates, each emitting a **PrivilegedUpdated** event. Inheriting contracts must implement their own access control on the setter functions. Both concrete implementations restrict them to **onlyManagement**. The **isPrivileged** view function is publicly accessible for any external contract or caller to check an address's privileged status.

Core Invariants:

INV 1: Any address that has been granted privileged status must be able to have it revoked.

Issue_15	<code>PrivilegedUpdated</code> can emit an event without a state change
Severity	Informational
Description	The <code>_setPrivileged</code> and <code>_setPrivilegedBatch</code> internal functions in <code>Privileged</code> always assign the privileged mapping entry for each target address, then emit <code>PrivilegedUpdated</code> with the requested flag. Neither path reads the prior stored value before the write. A call that sets an address to true when it is already true or to false when it is already false leaves storage unchanged but still emits the event. Off-chain indexers and monitors that treat each <code>PrivilegedUpdated</code> log as a fresh grant or revocation, therefore observe spurious updates and skewed timelines relative to actual privilege changes.
Recommendations	Consider loading the current privileged flag before writing in <code>_setPrivileged</code> and <code>_setPrivilegedBatch</code> assign and emit <code>PrivilegedUpdated</code> only when the new status differs from the stored value.
Comments / Resolution	Fixed following recommendations.

YieldDonatingTokenizedStrategy

`YieldDonatingTokenizedStrategy` is a specialized `TokenizedStrategy` implementation designed for productive assets with discrete harvesting, where profits are donated by minting shares to the dragon router at the time of each report. It serves as the delegatecall implementation for yield-donating strategies, including `ERC4626Strategy`, `SparkStrategy`, `AaveV3Strategy`, and `YearnV3Strategy`, handling the conversion of realized yield into donation shares.

The contract overrides the report function to compare new total assets (from `harvestAndReport`) against the previously recorded total assets. When profits are detected, it converts the profit amount to shares and mints them directly to the dragon router. When losses occur, and the burning mechanism is enabled, it burns dragon router shares (up to the available balance) equivalent to the loss amount using floor rounding, with any residual loss reducing the price per share for all holders. Dragon router changes follow the inherited `TokenizedStrategy` cooldown and two-step finalization process.

Privileged Functions

- `report`

Core Invariants:

INV 1: Profit shares minted to the dragon router must correspond to the actual asset profit amount via `convertToShares`.

INV 2: Dragon share burning during loss protection can never exceed the dragon router's current share balance.

INV 3: When burning is disabled, losses are fully socialized across all shareholders via price-per-share reduction.

Issue_16	A malicious keeper can mess with the vault profits/losses
Severity	Governance
Description	A malicious or compromised keeper can strategically time report calls to extract value for the dragon router. By calling report during a temporary dip in the underlying yield source, the keeper materializes a loss that burns dragon shares and reduces PPS – then, when the position recovers, the next report treats the recovery as new profit and mints fresh shares to the dragon router, capturing value that should have remained with depositors. Conversely, the keeper can delay reporting during actual losses to let users exit at a stale (inflated) PPS, socializing the loss among remaining holders.
Recommendations	Consider acknowledging and documenting this issue.
Comments / Resolution	Acknowledged.

Issue_17	Dust share state can break share accounting, strand assets, and let the first depositor extract value from later depositors
Severity	Medium
Description	A dangerous accounting state can be reached in the yield-donating vault when a loss is realized while a user leaves behind a dust amount of shares. The root cause is the interaction between the withdraw path, the report path, and the special conversion behavior when supply or assets reach zero. The withdraw path burns shares based on the requested asset amount, not on the holder's full remaining balance. A sole holder can front-run a pending loss report and withdraw the full amount that is still recoverable under stale pre-loss accounting, while leaving behind a residual dust share balance. When the latter report recognizes the remaining loss, tracked totalAssets can fall to zero while totalSupply remains greater than zero. That already places the vault into a broken state. Deposits and mints are effectively DoSed there because share conversion returns zero when supply is nonzero and tracked assets are zero. The more serious problem appears if value later re-enters the strategy through direct donation of underlying, donation of external vault shares, recovery of previously impaired external shares, or emergency unwinding. In that recovery state, report measures a positive newTotalAssets while the old tracked totalAssets is still zero. The dragon mint path converts profit to shares through the normal conversion helper, and because supply is still nonzero while old tracked assets are zero, the conversion returns zero. The result is that the recovered value is not assigned to dragon at all. Instead, tracked totalAssets is updated while the old dust shares remain outstanding. At that point, the residual dust holder controls the entire claim on the recovered vault value. A further small withdrawal can burn the final dust share because withdraw rounds shares up, while only subtracting the small requested asset amount from tracked totalAssets. This leaves the vault in an inconsistent state where

`totalSupply` is zero while `totalAssets` is still positive. That state is extremely dangerous because the next depositor is treated as the first depositor and is minted shares at a 1 to 1 ratio, even though the vault already holds assets.

This creates a direct extraction path. Assume the vault has reached `totalAssets` of 100e18 with `totalSupply` equal to zero. A victim submits a deposit of 150e18, expecting the normal first-depositor behavior. An attacker sees the transaction in the mempool and front-runs it with a dust deposit of 1 wei. Because supply is zero, the attacker receives 1 wei share. The vault now holds 100e18 assets and 1 wei total shares. When the victim deposit executes next, share minting is floor-rounded against that extreme ratio, so the victim receives only 1 wei share instead of a fair proportional amount. After the victim deposit, the vault holds 250e18 assets against 2 wei total shares, so each wei share is worth 125e18. The attacker can then redeem their 1 wei share for 125e18, turning a dust deposit into a claim on half of the vault, while the victim's 150e18 deposit is immediately worth only 125e18. This is not just a stranded asset edge case. Once the vault reaches `totalSupply` equal to zero while `totalAssets` remains positive, the first post-reset deposit becomes a mempool race, and later depositors can be diluted through truncation.

The damage does not stop there. Once the supply is reduced to dust while assets remain large, most later profit reports in the yield-donating flow will also mint zero dragon shares because the profit-to-share conversion truncates to zero at the extreme price per share. In that state, future value accrues to the dust holder rather than to dragon. This means the donation mechanism itself can effectively stop working after the accounting state is corrupted.

In the concrete ERC4626Strategy and YearnV3Strategy wrappers, recovery can be even harder to reconcile because BaseHealthCheck may revert the report that would otherwise restore tracked `totalAssets` after a jump from zero to positive assets. That turns the issue from an accounting distortion into an operational dead end unless management intervenes.

Recommendations	If the protocol maintains a small position, the risk of dust-based accounting attacks almost drops to 0.
Comments / Resolution	Fixed by minting the first 1000 shares to a dead address and adding safeguards to the max deposit/redeem functions.

Issue_18	Share inflation possible during large losses
Severity	Medium
Description	In case of a large loss where users exit the vault before losses get reported, the protocol can enter a state where very few wei of assets back a large number of shares. Consider the vault has 1000e18 DAI of backing, and there is a loss of 200e18 DAI, with 1:1 share ratio. Users can quickly withdraw all except 1 wei of assets by burning up ~800e18 shares, leaving the pool with 1 wei DAI backing and 200e18 shares. Now if users enter the vault at this state, they will be minted 200e18 shares for each wei of their deposit, meaning a simple 1e18 DAI deposit can net 200e36 shares. Such inflated share numbers can lead to overflows in vault operations, leading to stuck funds. The contract implements safeguards for the scenario where the <code>totalAssets</code> hit 0, but the share inflation can happen if the <code>totalAssets</code> hit any low number, not just 0.
Recommendations	A clean fix for this is difficult. In case of very large losses putting the vault in this state, consider shutting them down and creating a new vault with the share ratios reset to avoid large share inflation. Comment from Supervisor: We recommend carefully elaborating the approach of making further changes, as historically speaking, with each new change/iteration of the code, it is very likely to not only introduce isolated edge-cases to the corresponding spot at the code but also context-related edge-cases which might not appear obvious to explore. Further changes should be a joint-effort between auditors and developers and should then be followed by not only an isolated change review but also a contextual review which covers potentially broader impacts.
Comments / Resolution	Acknowledged – mitigation is operational: should the vault land in this state following a very large underlying loss, management will trigger <code>shutdownStrategy</code> and redeploy a fresh vault rather than attempt in-place recovery.

Issue_19	Asymmetric downside leads to an ever-decreasing share value
Severity	Medium
Description	In <code>YieldDonatingTokenizedStrategy</code>, <code>report</code> compares the harvested <code>newTotalAssets</code> to the stored total before the update. Whenever the balance is higher, the full positive gap is treated as profit and converted to shares that are minted to the dragon router. There is no branch that earmarks part of that upside to make depositors whole after a prior impairment. On the loss leg, <code>_handleDragonLossProtection</code> may burn dragon shares up to the loss when burning is enabled, but anything it does not cover reduces <code>totalAssets</code> and therefore price per share for all holders. The implementation does not keep a separate loss bucket to amortize back through future reports. So the economics are asymmetric in a way that is easy to under-read from the outside. Downside that slips past the dragon lands permanently in depositor PPS. The next time the strategy prints a gain, that gain still routes entirely to the dragon mint path instead of first refilling the hole in depositor value. Over a sequence of shocks and recoveries, share value can keep stepping down while headline profit events keep benefiting the router, even when the underlying position has regained notional size.
Recommendations	Document that the uncovered loss is permanent in strategy and that there is no route to recovering prior PPS.
Comments / Resolution	Acknowledged.

Issue_20	<code>totalAssets == 0 && totalShares > 0</code> will DOS the contract unless donations are made
Severity	Low
Description	In case <code>totalAssets == 0 && totalShares > 0</code> , which can occur following a loss report, both deposit and mint will revert. The only way to make the strategy function again is through donations, which will go directly to the shareholder rather than the dragon router, since minted profit shares will be 0.
Recommendations	Maintain a small position owned by the protocol or mint some shares to the dead address.
Comments / Resolution	Acknowledged.

Issue_21	Preview functions do not take into account solvency or dragon share burning
Severity	Low
Description	According to ERC4626, the preview functions [previewMint, previewRedeem, etc.] are supposed to accurately predict the results of the mint/redeem operations. However, this is not the case here, since the preview functions simply do asset - share conversions. The actual mint/redeem logic has other criteria that are not implemented in the preview functions, like vault insolvency in case of skimming strategies or dragon share burning logic in all strategies. This breaks the ERC4626 standard, since the preview functions are inaccurate.
Recommendations	Consider overriding the preview functions to accurately calculate the results.
Comments / Resolution	Partially fixed. Preview redeem functions in yield donating strategies do not simulate dragon share burns.

Issue_22	Zero-share donation emits on dust profit
Severity	Informational
Description	<code>YieldDonatingTokenizedStrategy</code> report converts positive profit in assets to dragon shares using Floor rounding. When profit is positive but below the threshold that maps to one full share, given the current total supply and total assets, <code>sharesToMint</code> becomes zero. The branch still calls <code>_mint</code> with zero and emits <code>DonationMinted</code> with zero amount. Off-chain indexers observe a donation event without a supply change, and the call pays for a no-op mint.
Recommendations	Skip <code>_mint</code> and <code>DonationMinted</code> when rounded profit shares are zero, or document dust handling so pipelines treat zero-amount donation events as non-events.
Comments / Resolution	Fixed following recommendation.

PrivilegedYieldDonatingTokenizedStrategy

`PrivilegedYieldDonatingTokenizedStrategy` is a whitelist-gated extension of `YieldDonatingTokenizedStrategy` that restricts deposit and mint operations to addresses marked as privileged by management. It is used when the Octant system requires controlled access to specific yield-donating strategy vaults, ensuring only approved addresses can deposit assets into or mint shares from the strategy. It inherits from both `YieldDonatingTokenizedStrategy` and the `Privileged` contracts.

Privileged Functions

- `setPrivileged`
- `setPrivilegedBatch`
- `deposit`
- `mint`

Core Invariants:

- INV 1: Only addresses marked as privileged by management can deposit assets.
- INV 2: Only addresses marked as privileged by management can mint shares.
- INV 3: Both the caller and the receiver must be privileged for any `deposit` or `mint` to `succeed`.
- INV 4: Withdrawals and redemptions are not restricted by privileged status.
- INV 5: `maxDeposit` and `maxMint` return 0 for non-privileged addresses.

Issue_23	Any address can hold tokens
Severity	Informational
Description	The deposit function makes sure that only a privileged address can mint tokens or receive them. <pre>require(isPrivileged(msg.sender) && isPrivileged(receiver), "!privileged");</pre> While this makes sure that the receiver is also privileged, it cannot really control which address ends up actually holding the tokens. This is because the strategy contract mints simple ERC20 tokens to the receiver, which can be moved around with the transfer function. The transfer function does not check for privileged addresses, so while tokens can only be minted to privileged addresses, they can then be transferred around and held by any address.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

YieldSkimmingTokenizedStrategy

`YieldSkimmingTokenizedStrategy` is a specialized `TokenizedStrategy` implementation designed for yield-bearing assets with appreciating exchange rates such as `wstETH` or `rETH`, where shares represent `ETH` value rather than asset amounts. It captures yield by comparing the current total vault value (assets multiplied by the exchange rate) against total outstanding share debt, minting dragon router shares for excess value and burning them to absorb losses. It is used as the `delegatecall` implementation for yield-skimming strategies like `LidoStrategy` and `RocketPoolStrategy`.

The contract maintains dual-mode conversion: when solvent, it uses rate-based conversions [`assets * exchangeRate`] for share pricing; when insolvent (vault value below user debt), it falls back to proportional distribution using the base `TokenizedStrategy` logic. Dragon router shares serve as a loss-absorption buffer; they are burned first during losses when `enableBurning` is true, and cannot be redeemed if doing so would compromise user debt coverage. Debt tracking is split between user debt and dragon router debt, with transfers between dragon and non-dragon addresses triggering debt rebalancing. The dragon router cannot deposit, mint, or transfer shares to itself, and all deposits and mints are blocked during insolvency.

Privileged Functions

- `report`

Core Invariants:

INV 1: The Dragon router cannot transfer shares to itself.

INV 2: Dragon router cannot deposit, or mint shares—dragon shares are only created via `report`.

INV 3: Dragon router cannot redeem or withdraw shares if doing so would leave the vault unable to cover user debt (when `enableBurning` is true).

INV 4: When the vault is insolvent, all deposits and mints are blocked.

Issue_24	Dragon router replacement can silently reduce loss-protection capacity without user notification.
Severity	Governance
Description	When management replaces the dragon router via <code>finalizeDragonRouterChange</code> , the new router may hold significantly fewer strategy shares than the old one, reducing or eliminating the vault's ability to absorb losses. Management is expected to either transfer existing shares to the new router, ensure it holds a comparable share balance before finalizing the change, or notify depositors in advance - the 14-day cooldown provides a window for this.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_25	All dragon solvency restrictions in <code>YieldSkimmingTokenizedStrategy</code> can be bypassed by toggling <code>enableBurning</code> off and on within the same block.
Severity	Governance
Description	Every function that restricts dragon router operations - <code>_requireDragonSolvency</code> , <code>_requireDragonSolvencyAfterOperation</code> , <code>_maxDragonRedeemableShares</code> , <code>_applyDragonLossProtectionIfNeeded</code> - returns early when <code>enableBurning = False</code> . Management controls <code>enableBurning</code> via a direct setter with no cooldown. In a single block, management can disable <code>enableBurning</code> , execute a dragon router operation that would otherwise be blocked (e.g. <code>redeem</code> beyond <code>maxDragonRedeemableShares</code> , operate during insolvency), then re-enable it - leaving the vault in a state that the solvency system was designed to prevent.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged. <code>enableBurning</code> can no longer be turned off if burns are pending.

Issue_26	Dust share transfer can block dragon router migration
Severity	Medium
Description	A holder can block dragon router migration by transferring a dust amount of shares to the current dragon router before finalization. The migration logic only enforces the old dragon solvency gate when the old dragon router holds a nonzero balance. If the current dragon has no shares, finalization skips that gate entirely and can proceed even while the vault is insolvent. This makes the migration outcome sensitive to trivial balance changes. By sending even a very small number of shares to the current dragon router, a user causes those shares to be reclassified into dragon debt and turns the old dragon balance from zero to nonzero. From that point onward, <code>FinalizeDragonRouterChange</code> starts enforcing the old-dragon solvency requirement. If the vault is insolvent, the migration now reverts. The result is a low-cost grieving vector. A migration that would otherwise succeed can be blocked by any holder willing to sacrifice a dust amount of shares. The attacker does not profit directly, but they gain the ability to interfere with router rotation during periods of stress, which is exactly when operational flexibility is most important.
Recommendations	The fix for this is not trivial; do not allow users to decide whether a migration is possible or not.
Comments / Resolution	Fixed by skipping the old dragon router solvency check. Loss protection is now applied before migration as long as <code>enableBurning</code> is true. Migration however can only proceed if the vault is solvent, if burning is enabled.

Issue_27	Vault solvency check is flawed when <code>enableBurning</code> is false
Severity	Low
Description	In case <code>enableBurning</code> is false, <code>dragonShares</code> are considered equivalent to <code>userShares</code>. But the vault solvency check doesn't include the <code>dragonRouterShare</code> when comparing against the total asset value. This will cause the vault to be incorrectly marked as solvent, allowing withdrawals of assets at higher than correct prices causing the other users to suffer losses Eg: <code>userShares</code> = 10 <code>totalAssets</code> = 15 <code>exchangeRate</code> = 1 <code>dragonRouterShare</code> = 10 <code>enableBurning</code> = false Since <code>userShares</code> < <code>totalAssets</code> * <code>exchangeRate</code>, vault is considered solvent and hence redemptions use <code>1/exchangeRate</code> as the share price (rather than using lower <code>totalAssets/totalSupply</code>) Dragon redeems 10 shares, <code>assetsToWithdraw</code> = 10 * 1 == 10 after withdrawal, <code>userShares</code> = 10 <code>totalAssets</code> = 5 <code>exchangeRate</code> = 1 <code>dragonRouterShare</code> = 0 Now the vault is insolvent, and each user receives only ½ asset worth 0.5 after redeeming per share
Recommendations	Include <code>dragonShares</code> as well in the vault solvency check if <code>enableBurning</code> is false.
Comments / Resolution	Acknowledged.

Issue_28	Exchange rate arbitrage via instant deposit/redeem
Severity	Low
Description	Deposits and redemptions in YieldSkimmingTokenizedStrategy are priced using the live exchange rate from the underlying protocol [stETH, rETH] with no commitment delay, snapshot, or cooldown between operations. A user can deposit at one rate and redeem at a different rate within the same block or across consecutive blocks, extracting assets from the vault. A user who deposits at rate R1 and redeems at rate R2 < R1 receives assets * R1 / R2 back – more than they put in, as long as the vault is solvent. An attacker monitoring for upcoming rate decreases (slashing events, oracle updates) can deposit just before the drop and redeem immediately after, profiting at the expense of existing depositors. This pushes the vault toward insolvency and can trigger dragon share burning. This is actually not possible normally. Rocketpool charges a 0.05% fee for native swaps, and Lido has an unstaking delay of 4-5 days. Both these mechanisms prevent instant profit capture, while the protocol here allows any user to use the exact exchange rate at no delay. This gradually pushes the vault closer and closer to insolvency, and every slashing event becomes profitable for MEV bots, erasing the profits from the dragon router.
Recommendations	Consider adding a delay for withdrawals or a deposit/withdrawal fee to prevent users from gaming the system via frictionless arbitrage.
Comments / Resolution	Acknowledged.

Issue_29	Deposits can be frequently blocked due to volatile exchange rates
Severity	Medium
Description	<code>YieldSkimmingTokenizedStrategy</code> marks the vault insolvent when total assets times the live exchange rate in value units falls below <code>totalDebtOwedToUserInAssetValue</code>. Deposit and mint both call <code>_requireVaultSolvency</code> and revert while that condition holds. After <code>report</code> skims surplus to the dragon router, accounting can sit at a tight breakeven between vault value and booked debt. A modest downward tick in the external rate, within ordinary feed noise, is enough to flip the inequality without a large economic shock. The vault then blocks new deposits until the rate later increases and realigns value and debt. Strategies whose reference rate has potential or a history of minor decreases, even if only temporarily, will be subject to frequent downtime for depositors.
Recommendations	Document that deposits revert under insolvency after tiny rate dips post-skim; advise keepers to monitor volatile reference rates and be considerate of such behavior when reporting. Otherwise, consider implementing a buffer, such that the dragon router cannot redeem all their shares and instead needs to keep some shares to absorb potential losses to keep the vault open.
Comments / Resolution	Acknowledged.

Issue_30	<code>finalizeDragonRouterChange</code> can mess with solvency operations
Severity	Medium
Description	If the vault is insolvent, the <code>finalizeDragonRouterChange</code> is reverted even if the router change would have made it solvent. If the vault is solvent, the router change is always approved, even if the change actually makes it insolvent. The root cause for both these issues is that the router change function only checks the solvency at the start of the operation instead of at the end. Assume the router holds 100 shares, users hold 100 shares, and the vault only has 150 assets. Since $150 > 100$, the vault is solvent, so a fresh new router can be set up. But if the router is changed, the router's 100 shares become user shares, and now there are 200 user shares for 150 assets, so the vault is insolvent.
Recommendations	Consider having the solvency check at the end.
Comments / Resolution	Fixed. A post migration solvency check has been added, and loss protection is triggered before migration.

Issue_31	Oracle frontrunners can profit from rate increments
Severity	Low
Description	Rocketpool updates its exchange rate once every 24 hours, and Lido updates its rates every day at 12 PM UTC. These are predictable timestamps, and users can game these updates using the frictionless exchange system in the contracts here for profit. Say the oracle is updating the rate from 1.05 to 1.051. Users can frontrun the update to redeem 100e18 shares to get back 95.238e18 base tokens and then immediately mint with the same funds to get 100.095e18 share tokens, netting them a profit of 0.095e18 tokens. This can be carried out predictably every day for profit, costing the dragon router, since it will not get minted any profit shares. A simple script can economically generate extra profit for users.
Recommendations	Consider charging a small deposit/withdrawal fee to prevent this kind of gaming of the system.
Comments / Resolution	Acknowledged.

Issue_32	Hindsight impact of <code>enableBurning</code>
Severity	Low
Description	When <code>enableBurning</code> is true, users expect the <code>dragonRouter</code> to cover losses first. Since <code>enableBurning</code> doesn't sync the current strategy value with existing user and dragon debts, it is possible for the strategy to be insolvent while having non-zero dragon shares when <code>setEnableBurning</code> is called. In this case, the dragon shares will now be able to redeem the assets pro rata with the user shares, although the losses were from a period when <code>enableBurning</code> was true.
Recommendations	Sync between the shares and strategy value (by invoking <code>report</code>) before <code>setEnableBurning</code> is called.
Comments / Resolution	Fixed by forcing <code>report</code> to be called before <code>enableBurning</code> is changed.

Issue_33	Dragon withdrawals are disabled in case <code>currentRate == 0</code>
Severity	Informational
Description	In case <code>currentVaultValue == YS.totalDebtOwedToUserInAssetValue</code> , <code>_maxDragonRedeemableShares</code> returns 0, i.e., dragon cannot withdraw any shares. This is because further withdrawals are assumed to make the vault insolvent. But this is not the case when <code>totalDebtOwedToUserInAssetValue == 0</code> and <code>currentRate == 0</code> . In this case, the vault owner's withdrawal of assets has no effect on the vault's solvency and should ideally be allowed
Recommendations	In case <code>currentRate == 0</code> and <code>totalDebtOwedToUserInAssetValue == 0</code> , return <code>YS.dragonRouterDebtInAssetValue</code>
Comments / Resolution	Acknowledged.

Issue_34	<code>_requireDragonSolvencyAfterOperation</code> check is flawed for withdrawals
Severity	Informational
Description	Instead of comparing if the new assets after dragon withdrawal will cover the user debt, <code>_requireDragonSolvencyAfterOperation</code> function rearranges the equation and checks if current assets will cover existing user debt + removal shares. While this is fine in the general case, it can be incorrect when <code>totalDebtOwedToUserInAssetValue</code> is 0, since then the vault should always be considered as solvent. If <code>totalDebtOwedToUserInAssetValue</code> and <code>currentExchangeRate</code> are both 0, but <code>totalAssets</code> != 0, dragon withdrawals will revert due to the above issue
Recommendations	Rewrite the formula to compare between subtracted <code>totalAssets</code> and <code>totalDebtOwedToUserInAssetValue</code> .
Comments / Resolution	Acknowledged.

Issue_35	<code>lastReportedRate</code> won't correspond to <code>lastReport</code>
Severity	Informational
Description	Ideally, both <code>lastReportedRate</code> and <code>lastReport</code> should correspond to values as of the latest report call. But before the report is invoked for the first time, these values can belong to different timestamps. <code>lastReport</code> will have the timestamp at which the strategy was initialized, and <code>lastReportedRate</code> can correspond to the exchange rate that was present during the first deposit
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_36	Reported loss does not differentiate between old and new losses
Severity	Informational
Description	When vault value falls below combined user and dragon debt, <code>YieldSkimmingTokenizedStrategy</code> report sets <code>lossValue</code> to that gap, and <code>_handleDragonLossProtection</code> converts the full gap to asset units for the return value and <code>Reported</code> event. Inline comments describe this as a gross loss before dragon absorption. The figure measures how far outstanding debt exceeds current value, not the incremental move since the prior report. If keepers invoke report again while the same impairment persists, the event replays essentially the same gross shortfall unless dragon burns or other state changes shrink the gap first. Integrations that sum loss fields across reports, therefore, count one sustained deficit multiple times and overstate cumulative damage.
Recommendations	Document in NatSpec that Reported loss is a level-style gross shortfall per call and must not be naively summed, or change reporting to emit only the delta since the last stored snapshot.
Comments / Resolution	Acknowledged.

issue_37	Donated assets to <code>YieldSkimmingTokenizedStrategy</code> are invisible to solvency checks until <code>report</code> is called.
Severity	Low
Description	All solvency and dragon-restriction logic in <code>YieldSkimmingTokenizedStrategy</code> derives vault value exclusively from <code>S.totalAssets</code>. This does not take into consideration any donated tokens present as raw balance: <pre><i>uint256 currentVaultValue = S.totalAssets.mulDiv(currentRate, WadRayMath.RAY)</i></pre> Until <code>report</code> runs, every function that uses <code>_isVaultInsolvent</code> operates on an understated vault value, since it only uses the recorded value, not the actual balance. A vault that is genuinely solvent (donated assets bring the real balance above user debt) can appear insolvent, blocking new deposits and dragon operations for no valid reason.
Recommendations	If silent donations are an expected operational pattern, <code>_isVaultInsolvent</code> could incorporate <code>S.asset.balanceOf(address[this])</code> as a floor for vault value assessment alongside <code>S.totalAssets</code> .
Comments / Resolution	Acknowledged.

Issue_38	Management's attempt to disable <code>enableBurning</code> during insolvency can be front-run by a user <code>redeem/withdraw</code>
Severity	Low
Description	When the vault becomes insolvent and <code>enableBurning = true</code>, the first user <code>redeem</code> or <code>withdraw</code> call triggers <code>_applyDragonLossProtectionIfNeeded</code>, which burns all dragon router shares up to the loss amount: <pre>// YieldSkimmingTokenizedStrategy.sol:161-163 if (owner != S.dragonRouter) { _applyDragonLossProtectionIfNeeded(S, YS); }</pre> If management wants to disable <code>enableBurning</code> to preserve the dragon router's share balance, they must do so before any user exit executes. On a public mempool, a user observing the management <code>setEnableBurning(false)</code> transaction can front-run it with a <code>redeem(1 share)</code> call, forcing <code>_applyDragonLossProtectionIfNeeded</code> to fire and irreversibly burn the dragon balance. Even a minimal redemption triggers the full lazy burn.
Recommendations	Protocol explicitly requires keepers and management to use MEV-protected mempools. On a private mempool this race condition cannot be triggered. Confirm all management operations on <code>enableBurning</code> are submitted exclusively through private relay infrastructure.
Comments / Resolution	Fixed by forcing <code>report</code> to be called before <code>enableBurning</code> is changed.

Issue_39	YIELD_SKIMMING_STORAGE_SLOT deviates from ERC-7201
Severity	Informational
Description	YieldSkimmingTokenizedStrategy computes its storage slot without the outer keccak256(abi.encode(...)) wrap and the & ~bytes32(uint256(0xff)) alignment mask required by ERC-7201.
Recommendations	Calculate YIELD_SKIMMING_STORAGE_SLOT with ERC-7201 formula
Comments / Resolution	Fixed following recommendations.

Issue_40	Non-saturating dragon debt subtraction
Severity	Informational
Description	In the <code>_handleDragonLossProtection</code> function, the <code>dragonRouterDebtInAssetValue</code> value is decreased by the <code>dragonBurn</code> amount. <code>dragonBurn</code> is capped to the actual balance. <pre>uint256 dragonBurn = Math.min(lossValue, dragonBalance); // {valueShare} _burn[S, S.dragonRouter, dragonBurn]; // update the dragon value debt YS.dragonRouterDebtInAssetValue -= dragonBurn; // {valueShare} - = {valueShare}</pre> This uses a raw subtraction and does not saturate to 0. So if the dragon balance ever exceeds <code>dragonRouterDebtInAssetValue</code>, this can lead to a revert. All other <code>dragonRouterDebtInAssetValue</code> manipulation actions use saturated subtraction, so this should as well.
Recommendations	Consider using saturated subtraction for <code>dragonRouterDebtInAssetValue</code> .
Comments / Resolution	Fixed following recommendation.

Issue_41	Excessive rounding versus dragon burn
Severity	Informational
Description	<code>report</code> uses floor rounding twice when measuring profit: <code>currentValue = totalAssets * rate / RAY [floored]</code> <code>profit = profitValue * RAY / rate [floored again]</code> But <code>profitShares = profitValue</code> is minted to the dragon router at full integer precision, no rounding applied. This means the reported profit in asset terms is slightly less than the value the dragon actually received in shares. Over many reports, the dragon router accumulates a tiny amount of value that wasn't reflected in the emitted profit figures.
Recommendations	Consider acknowledging this issue, or calculate profit with a single rounding operation.
Comments / Resolution	Acknowledged.

Issue_42	Yield skim docs implied ETH share peg
Severity	Informational
Description	NatSpec and inline comments on <code>YieldSkimmingTokenizedStrategy</code> and <code>IYieldSkimmingStrategy</code> described user and dragon debt as ETH value and stated one share equals one ETH value. The implementation books value debt in units tied to <code>getCurrentExchangeRate</code> on the strategy asset, so the reference tracks that underlying and its rate, not a fixed one-to-one mapping to native ETH. <code>wstETH</code> is intended to stay 1:1 with the liquid staking asset <code>stETH</code>, while this asset ought to stay close and strives to be equal to spot ETH. This is not always the case, so the wording misstates the accounting basis for integrators and reviewers.
Recommendations	Rewrite share and debt documentation so every occurrence states one share equals one unit of underlying asset value at the strategy exchange rate, not native ETH, across yield skimming core and interfaces.
Comments / Resolution	Fixed by editing documentation.

PrivilegedYieldSkimmingTokenizedStrategy

`PrivilegedYieldSkimmingTokenizedStrategy` is a whitelist-gated extension of `YieldSkimmingTokenizedStrategy` that restricts deposit and mint operations to addresses marked as privileged by management. It is used when the Octant system requires controlled access to yield skimming strategy vaults holding yield-bearing assets like wstETH or rETH, ensuring only approved addresses can deposit. It inherits from both `YieldSkimmingTokenizedStrategy` and the `Privileged` contracts.

Privileged Functions

- `setPrivileged`
- `setPrivilegedBatch`
- `deposit`
- `mint`

Core Invariants:

- INV 1: Only addresses marked as privileged by management can deposit assets.
- INV 2: Only addresses marked as privileged by management can mint shares.
- INV 3: Both the caller and the receiver must be privileged for any `deposit` or `mint` to succeed.
- INV 4: Withdrawals and redemptions are not restricted by privileged status.
- INV 5: `maxDeposit` and `maxMint` return 0 for non-privileged addresses.

No issues were identified in this contract.

BaseHealthCheck

`BaseHealthCheck` is an abstract safety layer that inherits `BaseStrategy` and adds profit/loss bounds checking to prevent unexpected or manipulated report values from being permanently recorded. It is used by all yield-donating strategies in the system, including `ERC4626Strategy`, `SparkStrategy`, `AaveV3Strategy`, and `YearnV3Strategy`, providing configurable guardrails against anomalous yield or loss events.

The contract intercepts the `harvestAndReport` function to compare the reported `totalAssets` against configurable profit and loss limit ratios denominated in basis points. If the reported profit exceeds the `profitLimitRatio` or the reported loss exceeds the `lossLimitRatio`, the transaction reverts, requiring management intervention to investigate before reporting. The health check can be temporarily disabled by management, after which it automatically re-enables on the next report.

Privileged Functions

- `setProfitLimitRatio`
- `setLossLimitRatio`
- `setDoHealthCheck`
- `harvestAndReport`

Core Invariants:

INV 1: If `doHealthCheck` is disabled, the next report automatically re-enables it.

Issue_43	Health check uses post-withdraw <code>totalAssets</code>
Severity	Low
Description	<code>BaseHealthCheck harvestAndReport</code> runs <code>_harvestAndReport</code> then <code>_executeHealthCheck</code>. The helper loads <code>TokenizedStrategy totalAssets</code> as the baseline for profit and loss ratio tests against the freshly harvested balance. After large withdrawals, the stored total can be a small remainder, while profit was earned based on a much larger amount. The check treats the balance delta as a return percentage of that remainder, so a 20 asset gain on a 100 asset economic history reads as a 200 percent move against 10 assets left and trips default profit limits even when return on deployed capital is modest. Furthermore, vault operations can also be temporarily DOSed via external donations. Donated funds are seen as profit, and if the donation amount makes the vault hit the <code>_profitLimitRatio</code>, the report call reverts. This will then delay yield reporting until admins intervene. Report then reverts to <code>healthCheck</code> until management calls <code>setDoHealthCheck</code> or widens ratios. <code>BaseYieldSkimmingHealthCheck</code> instead compares exchange rates, so this distortion targets asset-denominated strategies using <code>BaseHealthCheck</code>.
Recommendations	Use last-reported <code>totalAssets</code> or another basis that does not shrink from voluntary exits alone to determine health checks.
Comments / Resolution	Acknowledged.

Issue_44	NatSpec typo OVerrides on harvestAndReport
Severity	Informational
Description	BaseHealthCheck harvestAndReport NatSpec notice starts with OVerrides instead of Overrides.
Recommendations	Change the @notice line to use Overrides instead.
Comments / Resolution	Fixed following recommendations.

AAVEV3Strategy

AAVEV3Strategy is a yield-donating strategy that deposits assets into Aave V3 lending pools to earn interest, which is then donated through the **BaseHealthCheck** profit minting mechanism. It integrates with the Aave V3 protocol via the **PoolAddressesProvider** to dynamically resolve pool and data provider addresses, and derives the correct aToken from the pool registry at construction to ensure correctness. It is built as a concrete strategy inheriting **BaseHealthCheck**.

The contract manages deposits by supplying assets to the Aave V3 pool and receiving aTokens that automatically accrue interest over time. Deposit limits are dynamically calculated from the Aave V3 supply cap (in whole token units scaled by decimals) minus current total aToken supply and idle balance, while withdrawal limits account for available pool liquidity to prevent failures during high utilization periods. On report, total assets are calculated as the sum of aToken balance (1:1 value with underlying) and idle asset balance. Access control is inherited from **BaseHealthCheck** and **TokenizedStrategy**.

Core Invariants:

INV 1: Total reported assets must equal aToken balance plus idle asset balance.

INV 2: Withdrawal limit is bounded by the lesser of the strategy's aToken balance and pool liquidity, plus idle balance.

Issue_45	No claim path for Aave aToken incentives
Severity	Medium
Description	AaveV3Strategy only supplies and withdraws through the pool plus reads aToken balance in _harvestAndReport . The contract never integrates Aave rewards controller interfaces or any claim entry point for liquidity mining emissions tied to the supplied aToken position. Incentive balances accrue against this contract address on Aave but stay unclaimed because no function exists to move them into strategy custody. This leads to reward principal being locked inside Aave incentive accounting.
Recommendations	Add a management-gated claim and either pass the assets as is to a protocol-controlled address or swap the incentive for the strategy's asset and pass it on to the receiver.
Comments / Resolution	Fixed by adding functionality to claim/sweep tokens to the dragon router, which must implement ways to sweep or swap those tokens.

Issue_46	Aave deposit cap omits reserved balance
Severity	Low
Description	<code>AaveV3Strategy.availableDepositLimit</code> compares the scaled supply cap to aToken total supply from the data provider and returns the gap net of idle balance here. Aave, however, also tracks treasury reserve balances that consume cap headroom. <pre> [[IAToken(reserveCache.aTokenAddress).scaledTotalSupply() + uint256(reserve.accruedToTreasury)].rayMul(reserveCache.nextLiquidityIndex) + amount] <= supplyCap * (10 ** reserveCache.reserveConfiguration.getDecimals()), </pre> Thus, the treasury reserves also consume the cap in Aave, but this is not accounted for in the contract here, leading to over-reporting of the free cap in the system. This makes <code>maxDeposit</code> unreliable and will lead to unexpected reverts when any amount less than or equal to the incorrect deposit cap and greater than the true deposit cap is attempted.
Recommendations	Use the Aave treasury reserved balance against the cap headroom inside <code>availableDepositLimit</code> when a supply cap on Aave is being enforced.
Comments / Resolution	Partially fixed. The treasury amount is fetched from the data provider, however this isn't the updated data and thus differs from the values faced during deposit.

Issue_47	Deposit limit calculation doesn't consider possible reverts due to dust amounts
Severity	Low
Description	Dust amount can cause a revert during deposits if the corresponding share amount is 0 both in the vault and in AAVE. But the <code>maxDeposit</code> function of <code>AAVEV3Strategy</code> doesn't check this and can return dust values, which on actual deposit would revert (note that a similar issue is present in <code>YearnV3Strategy</code> as well).
Recommendations	Document this behaviour or check if the deposit will cause a revert.
Comments / Resolution	Acknowledged and documented.

Issue_48	Aave limit queries don't check the paused/frozen state
Severity	Low
Description	<code>AaveV3Strategy.availableDepositLimit</code> derives headroom from supply caps and aToken total supply, while <code>availableWithdrawLimit</code> blends aToken balance with underlying liquidity behind the aToken. Neither path queries the Aave reserve configuration for paused, frozen, or otherwise inactive states that block supply or withdrawal at the pool. When governance pauses or freezes a reserve, pool supply, and withdraw calls revert, while these views still return positive limits. Integrations and ERC4626 <code>maxDeposit</code> or <code>maxRedeem</code> style callers, therefore, see non-zero capacity and route users into transactions that fail at execution.
Recommendations	Consider checking the paused/frozen state of the Aave pool when checking for limits.
Comments / Resolution	Fixed following recommendations.

Issue_49	Withdrawal limit doesn't take into consideration the possible revert during the dust amount withdrawals
Severity	Low
Description	In case the <code>scaledAmount</code> corresponding to a withdrawal amount is 0, the withdrawal will revert in AAVE. But the <code>maxWithdraw</code> function doesn't check and still proceeds to return dust values that can cause a revert.
Recommendations	Document this behaviour or check if the withdrawal will revert.
Comments / Resolution	Acknowledged and documented.

Issue_50	<code>_emergencyWithdraw</code> may be limited due to aave pool pause or insufficient liquidity
Severity	Low
Description	<code>_emergencyWithdraw</code> delegates directly to <code>_freeFunds</code>, which calls <code>pool.withdraw</code> with no fallback: <pre>function _emergencyWithdraw(uint256 _amount) internal override { _freeFunds[_amount]; } // AaveV3Strategy.sol:191-193 function _freeFunds(uint256 _amount) internal override { pool.withdraw(address(asset), _amount, address(this)); }</pre> <code>_emergencyWithdraw</code> is called during strategy shutdown to pull funds back to idle so they can be returned to users. It is the last line of defence when something has gone wrong. Yet it will revert in two conditions that are most likely to coincide with an actual emergency: Pool paused - Aave's guardian can pause a reserve in response to an exploit or oracle failure; Insufficient pool liquidity - high utilization scenario leaves less liquidity in the aToken contract than the strategy holds. After the call reverts, the strategy remains invested in the troubled position with no automated way to exit.
Recommendations	Consider acknowledging this issue and taking care before calling emergency shutdowns, since recovery depends on liquidity conditions.
Comments / Resolution	Acknowledged and documented.

Issue_51	Max approval exposes full idle balances
Severity	Low
Description	AaveV3Strategy, YearnV3Strategy, and ERC4626Strategy each set <code>forceApprove</code> on the strategy asset to maximum uint256 allowance toward the external Aave pool, Yearn vault, or target ERC4626 vault during construction. A compromised counterparty or hostile upgrade at that vault can transfer the idle asset balance the strategy holds under that standing approval without a new governance action between events. This risk is not necessary as approval can be granted and revoked on each transfer instead. With a minor increase in gas spending, the protocol is less exposed to 3rd-party security risks.
Recommendations	Grant exact amounts when approval is needed and set approval to 0 after action is completed.
Comments / Resolution	Fixed following recommendation.

Issue_52	Immutable <code>dataProvider</code> misses Aave upgrades
Severity	Low
Description	<code>AaveV3Strategy</code> stores the pool and <code>dataProvider</code> addresses in immutable variables in the initialize function, derived from the <code>addressesProvider</code> address. These are, however, never updated. Thus, if the <code>addressesProvider</code> contract points to a different pool or a different <code>dataProvider</code> contract with different limits, this contract will keep using outdated limits since the data address will be stale, which can lead to unexpected reverts.
Recommendations	Consider reading the <code>pool</code> and <code>dataProvider</code> addresses directly from the <code>addressProvider</code> contract in every transaction, or introduce an admin-only function which updates the <code>pool</code> and <code>dataProvider</code> addresses to new values by querying the <code>addressProvider</code> .
Comments / Resolution	Fixed following recommendation.

ERC4626Strategy

ERC4626Strategy is a concrete yield-donating strategy that deposits assets into any ERC-4626 compliant vault to earn yield, which is then donated through the **BaseHealthCheck** profit minting mechanism.

The contract operates through a straightforward **deposit/withdraw** lifecycle against the target vault, with **_deployFunds** depositing into the vault and **_freeFunds** withdrawing from it. Deposit and withdrawal limits are dynamically derived from the target vault's own **maxDeposit** and **maxWithdraw** functions, adjusted for idle balance. On report, the strategy calculates total assets as the sum of its vault share value (via **convertToAssets**) and idle balance, with health check validation ensuring reported values stay within acceptable bounds. Access control is inherited from **BaseHealthCheck** and **TokenizedStrategy** via the fallback delegatecall.

Core Invariants:

INV 1: Available withdrawal limit is bounded by the target vault's **maxWithdraw** plus idle balance.

INV 2: Total reported assets must equal target vault share value (via **convertToAssets**) plus idle asset balance.

INV 3: Available deposit limit cannot exceed what the target vault accepts minus idle balance.

Issue_53	maxMint can overflow due to incorrect handling of a special case uint.max value inside availableDepositLimit
Severity	Low
Description	In case the underlying vault returns uint.max as the deposit limit, availableDepositLimit proceeds to subtract the current idle balance, returning a value that is still close to uint.max . This can cause overflow during _convertToShares inside _maxMint in case share value is greater than 1 (note that same issue is present in YearnV3Strategy as well).
Recommendations	Return uint.max as the availableDepositLimit in case the underlying vault has uint.max as the deposit limit.
Comments / Resolution	Fixed following recommendations.

Issue_54	Max approval exposes full idle balances
Severity	Low
Description	AaveV3Strategy, YearnV3Strategy, and ERC4626Strategy each set forceApprove on the strategy asset to maximum uint256 allowance toward the external Aave pool, Yearn vault, or target ERC4626 vault during construction. A compromised counterparty or hostile upgrade at that vault can transfer the idle asset balance the strategy holds under that standing approval without a new governance action between events. This risk is not necessary as approval can be granted and revoked on each transfer instead. With a minor increase in gas spending, the protocol is less exposed to 3rd-party security risks.
Recommendations	Grant exact amounts when approval is needed and set approval to 0 after action is completed.
Comments / Resolution	Fixed following recommendations.

Issue_55	ERC4626Strategy._harvestAndReport overstates totalAssets when targetVault charges exit fees
Severity	Low
Description	_harvestAndReport computes strategy value as convertToAssets(shares): <pre>uint256 shares = IERC4626(targetVault).balanceOf(address(this)); uint256 vaultAssets = IERC4626(targetVault).convertToAssets(shares);</pre> Per ERC4626, convertToAssets returns the gross asset value of shares and explicitly must not include fees. This means the reported totalAssets includes value that will never be realised - the portion that will be paid as exit fees. The strategy's position is systematically overvalued by $\text{shares} \times \text{exitFeeRate}$, inflating PPS for all depositors and causing the profit reported to YieldDonatingTokenizedStrategy to include phantom yield that cannot be withdrawn.
Recommendations	Document that ERC4626Strategy must not be paired with target vaults that charge exit fees. Otherwise, for asset valuation, better to use previewRedeem.
Comments / Resolution	Fixed following recommendations. The fix now expects the target vault's previewRedeem function to be properly implemented, including fees.

Issue_56	Lack of check <code>shares != 0</code> check on the underlying strategy deposit
Severity	Low
Description	During the <code>_deployFunds</code> strategy, the user's deposit is transferred to the underlying 4626 strategy. <pre><i>function _deployFunds(uint256 _amount) internal override { IERC4626(targetVault).deposit(_amount, address(this)); }</i></pre> If the deposit is too small, or the pps in the underlying strategy is high (due to an inflation attack or simply a high yield), then in the worst-case scenario, our strategy may receive 0 underlying shares for such a deposit.
Recommendations	Check that the number of shares received is not equal to 0. If it is, simply skip the deposit; the funds will be added along with the next deposits.
Comments / Resolution	Fixed by requiring minimum 1 share to be minted. Will revert if the underlying vault mints 0 shares for very small deposits.

Issue_57	Share rounding is seen as a loss, burning shares from the dragon router
Severity	Low
Description	ERC-4626 vaults store user positions in shares instead of the actual assets deposited. Users are minted a rounded-down number of shares. This means that immediately after a user deposits some tokens, they actually have control over fewer tokens due to the rounding down during share minting and another rounding down when converting the shares to underlying assets. The protocol, however, records the exact deposit value in <code>S.totalAssets</code>. Thus, a user can deposit 1000 tokens to a vault, and get minted shares which only represent 999 tokens (due to the rounding), but <code>S.totalAssets</code> will still reflect 1000. This is then treated as a loss, since the backing is now below the expected <code>totalAssets</code>, burning shares from the <code>dragonRouter</code>. Thus, even under normal operations, deposits can incur a loss, burning dragon shares, simply because of how 4626 vaults operate.
Recommendations	Consider acknowledging and documenting this behaviour.
Comments / Resolution	Acknowledged.

Issue_58	No checks on external vault deposit outputs
Severity	Informational
Description	Concrete strategies override BaseStrategy <code>_deployFunds</code> and <code>_freeFunds</code> by calling external vault deposit and withdraw helpers, then discarding every return value. ERC4626Strategy forwards full amounts to <code>IERC4626 deposit</code> and <code>withdraw</code> without asserting that minted shares or received assets stay positive. A malicious or broken downstream vault that honors the call without reverting but credits zero shares or pays zero assets drains inventory, while <code>TokenizedStrategy</code> already recorded the user deposit and minted Octant shares against the nominal asset input.
Recommendations	Consider acknowledging this issue or validating that each vault <code>deposit</code> and <code>withdraw</code> return is non-zero.
Comments / Resolution	Fixed by requiring minimum 1 share to be minted. Will revert if the underlying vault mints 0 shares for very small deposits.

Issue_59	Users can be drained in case target vault is fee capturing since entire value is credited to new depositors
Severity	Informational
Description	On a new deposit, the entire deposit amount is added to totalAssets and the amount of shares credited to the user is also calculated with this entire deposit amount. The fee that will be incurred on this amount during withdrawals only gets reflected when the report function is invoked. This is flawed as during the time gap b/w the deposit and the next report call, the user is allowed to withdraw the same amount that they had deposited while the target vault only allows withdrawal of [deposited amount - fee] on this deposit. The excess "fee" amount given to the new depositor comes from the existing users assets, hence draining them Eg: currently totalAssets = 0 totalShares = 0 targetVaultFee = 10% targetVaultAssets = 0 targetVaultShares = 0 1st user deposits 50 assets, totalAssets = 50 totalShares = 50 targetVaultAssets = 50 targetVaultShares = 50 2nd user also deposits 50 assets, totalAssets = 50 + 50 = 100 totalShares = 50 + 50 = 100 targetVaultAssets = 50 + 50 = 100 targetVaultShares = 50 + 50 = 100 2nd is now allowed to withdraw 50 assets itself but in the targetVault this will redeem 50/0.9 shares [since fees is captured on top of the withdrawal amount] leaving only ~ [100 - 55] = 45 assets and shares left totalAssets = 50 totalShares = 50 targetVaultAssets = 45 targetVaultShares = 45

	the 1st user will only be able to withdraw $45 * 0.9 \approx 40$ assets Please note that this issue is currently only rated as informational.
Recommendations	Currently, this does not expose an issue because such type of vaults are not supported. If that is ever planned, this issue would translate into a critical severity. The team confirms that they are aware of this constraint.
Comments / Resolution	Acknowledged. Exit fee vaults will not be used, added a Natspec comment reflecting the same.

SparkStrategy

SparkStrategy is a specialized yield-donating strategy that extends **ERC4626Strategy** specifically for Spark Protocol **ERC-4626** vaults, adding the ability to sweep airdropped governance tokens or other reward tokens to the dragon router address. It is designed for use within the Octant system to capture both lending yield from Spark and any additional token rewards that Spark Protocol may distribute to strategy addresses.

The contract inherits the full **ERC-4626** vault **deposit/withdraw** lifecycle from **ERC4626Strategy** and adds a single **sweepAirdrop** function that allows keepers or management to transfer non-critical airdropped tokens to the dragon router address. The sweep function explicitly prevents sweeping the strategy's main asset or target vault shares, ensuring operational integrity is maintained.

Privileged Functions

- **sweepAirdrop**

Core Invariants:

INV 1: The strategy's main asset token can never be swept via **sweepAirdrop**.

INV 2: The target vault's share token can never be swept via **sweepAirdrop**.

INV 3: Swept tokens are always sent to the current dragon router address.

Issue_60	Airdrop sweep is unsuited for the forwarder dragon
Severity	Informational
Description	<code>SparkStrategy</code> sweepAirdrop reads <code>TokenizedStrategy</code> dragonRouter, then safeTransfers the full airdrop balance to that address with no follow-up hook. YieldForwarder and <code>SwappingYieldForwarder</code> contracts hold fixed receiver logic for a single asset pipeline and expose no generic sweep for arbitrary ERC20 receipts. Tokens that are not the forwarder-configured asset sit idle with no callable path to reach the ultimate beneficiary. EOA dragon routers and payment splitter style recipients keep working because they either custody funds directly or forward through split rules. Deployments that point dragonRouter at an immutable forwarder therefore strand every swept airdrop inside the forwarder balance permanently.
Recommendations	Document <code>SparkStrategy</code> so <code>dragonRouter</code> is an EOA or splitter that accepts arbitrary ERC20 airdrops and avoids yield forwarders that cannot sweep non-pipeline tokens.
Comments / Resolution	Fixed by adding token recovery functions in the forwarder contracts.

YearnV3Strategy

YearnV3Strategy is a yield-donating strategy that deposits assets into a Yearn V3 vault to earn compounded yield, which is then donated through the **BaseHealthCheck** profit minting mechanism. It is designed for the Octant system to capture yield from Yearn's multi-strategy vaults, where the underlying vault may itself deploy into further meta-vaults, creating a multi-hop yield chain [e.g., Yearn vault to Morpho vault to underlying market].

The contract manages deposits and withdrawals against the Yearn V3 vault. Deposit limits are derived from the Yearn vault's **maxDeposit**, though these may be inflated for meta-vault configurations with duplicate markets in supply queues. On **report**, total assets are calculated as vault shares converted to assets plus the idle balance. Access control is inherited from **BaseHealthCheck** and **TokenizedStrategy**.

Core Invariants:

INV 1: Total reported assets must equal Yearn vault share value (via **convertToAssets**) plus idle asset balance.

INV 2: Available withdrawal limit is bounded by Yearn vault's **maxWithdraw** plus idle balance.

Issue_61	YearnV3Strategy._emergencyWithdraw hardcodes <code>maxLoss = 10_000</code> , removing the emergency admin's ability to enforce a loss limit
Severity	Low
Description	<code>_emergencyWithdraw</code> delegates directly to <code>_freeFunds</code>, which passes <code>maxLoss = 10_000</code> (100%) hardcoded to the Yearn vault: <pre>function _emergencyWithdraw(uint256 _amount) internal override { _freeFunds[_amount]; } ITokenizedStrategy[yearnVault].withdraw[_amount, address(this), address(this), 10_000];</pre> The 100% <code>maxLoss</code> in <code>_freeFunds</code> is intentional for normal operations - the comment explains that loss validation is delegated to <code>MultistrategyVault.updateDebt</code>. However, <code>_emergencyWithdraw</code> is called during shutdown: a fundamentally different context where the Strategy loss enforcement may not apply and where the emergency admin should be able to decide how much loss is acceptable before pulling funds. By routing through <code>_freeFunds</code>, the emergency admin is forced to accept any loss the Yearn vault reports, with no ability to abort if the loss exceeds an acceptable threshold.
Recommendations	Implement <code>_emergencyWithdraw</code> independently from <code>_freeFunds</code> with a configurable <code>maxLoss</code> parameter passed from <code>emergencyWithdraw(uint256 amount)</code> , or expose a separate <code>emergencyWithdraw(uint256 amount, uint256 maxLoss)</code> overload for the emergency admin.
Comments / Resolution	Acknowledged.

Issue_62	Max approval exposes full idle balances
Severity	Low
Description	AaveV3Strategy, YearnV3Strategy, and ERC4626Strategy each set forceApprove on the strategy asset to maximum uint256 allowance toward the external Aave pool, Yearn vault, or target ERC4626 vault during construction. A compromised counterparty or hostile upgrade at that vault can transfer the idle asset balance the strategy holds under that standing approval without a new governance action between events. This risk is not necessary as approval can be granted and revoked on each transfer instead. With a minor increase in gas spending, the protocol is less exposed to 3rd-party security risks.
Recommendations	Grant exact amounts when approval is needed and set approval to 0 after action is completed.
Comments / Resolution	Fixed following recommendations.

YieldForwarder

YieldForwarder is an immutable, single-purpose contract that automates the yield forwarding pipeline by calling **report** on a strategy, redeeming any resulting profit shares, and transferring the underlying assets to a hardcoded receiver address. It is designed to serve as both the strategy's keeper (authorized to call **report**) and the donation address (recipient of minted profit shares), creating a trust-minimized yield flow from strategies to the designated receiver.

The contract operates through a single atomic function, **reportAndForward**, which triggers a strategy report, redeems all shares held by this contract, and forwards the redeemed assets to the immutable receiver. If the report produces no profit shares, the function returns zero without reverting, ensuring the report itself can still serve loss accounting purposes. The contract is fully immutable with no admin functions, no upgrade path, and no token sweep capability—assets can only flow to the hardcoded receiver. Access is restricted to a single designated keeper address set at construction.

Privileged Functions

- **reportAndForward**

Core Invariants:

INV 1: All redeemed assets can only flow to the hardcoded receiver—there is no other destination.

Issue_63	Donated dust share reverts <code>TokenizedStrategy</code> reporting
Severity	Medium
Description	<code>TokenizedStrategy</code> redeem rejects any conversion that floors to zero assets through the <code>ZERO_ASSETS</code> guard after <code>_convertToAssets</code>. When stored <code>totalAssets</code> sits below total supply, the pro rata product for a single wei of share rounds down to zero assets. Anyone can transfer one strategy share into the forwarder so its balance is nonzero. <code>reportAndForward</code> and <code>reportSwapAndForward</code> call report, then redeem the full IERC20 balance in the same transaction. redeem reverts on that dust notional, so the whole outer call rolls back, and the keeper side report work never commits on the chain. When <code>totalAssets</code> is less than <code>totalSupply</code>, and dragon router burning is not enabled, an attacker can prevent losses from being realized by donating 1 wei worth of shares to the forwarder. This results in a persisting stale share value that can be exploited to escape losses. When used with the <code>YieldSkimmingTokenizedStrategy</code> in the same situation, an attacker can prevent reporting, impacting off-chain integrators that rely on such data.
Recommendations	Preview <code>convertToAssets</code> on the forwarder share balance in both forwarders, skip redeem when the preview is zero, so the keeper call still succeeds after the report.
Comments / Resolution	Fixed following recommendations.

Issue_64	Balance redeem ignores <code>maxRedeem</code> ceiling
Severity	Low
Description	The <code>reportAndForward</code> function in <code>YieldForwarder</code> and <code>reportSwapAndForward</code> in <code>SwappingYieldForwarder</code> both set shares from IERC20 <code>balanceOf</code> on this contract, then pass that amount into <code>redeem</code>. They never read <code>maxRedeem</code> for this contract from the strategy. Because the <code>availableWithdrawLimit</code> function can return a value less than the dragon router's actual balance, the attempt to redeem the full balance will revert, and the keeper loses a clean report plus the forward cycle. Repeated attempts keep failing until balance and <code>maxRedeem</code> realign through separate burns transfers or configuration work outside the forwarder path.
Recommendations	In <code>reportAndForward</code> and <code>reportSwapAndForward</code> replace raw balance with <code>maxRedeem</code> .
Comments / Resolution	Fixed following recommendations.

Issue_65	<code>sweepAirdrop</code> sends airdrop tokens to <code>dragonRouter</code> , locking them when <code>YieldForwarder</code> is the keeper
Severity	Informational
Description	<code>sweepAirdrop</code> always sends swept tokens to the current <code>dragonRouter</code> address: <pre><i>address dragon = TokenizedStrategy.dragonRouter{}; IERC20[_token].safeTransfer(dragon, balance);</i></pre> When <code>YieldForwarder</code> is deployed as the <code>dragonRouter</code>, the intended configuration for yield forwarding, it becomes the recipient of any swept airdrop tokens. <code>YieldForwarder</code> is a fully immutable contract with no admin, no token rescue function, and no sweep capability. Its only outbound transfer path is <code>reportAndForward</code>, which redeems strategy shares and forwards the underlying asset to the receiver. Arbitrary ERC-20 tokens sent to it have no exit path and are permanently locked.
Recommendations	Add an explicit recipient parameter to <code>sweepAirdrop</code> so the caller can direct airdrop tokens to an address that can actually handle them, instead of hardcoding <code>dragonRouter</code> .
Comments / Resolution	Fixed by adding a <code>forwardToken</code> function to rescue tokens.

Issue_66	YieldForwarder as dragonRouter silently disables enableBurning loss protection
Severity	Low
Description	report implements a loss protection mechanism: when a loss is detected, the strategy burns shares from the dragonRouter balance to absorb the loss on behalf of regular depositors: This mechanism requires the dragonRouter to hold an accumulated balance of shares. Shares are minted to it on every profitable report. YieldForwarder must be the dragonRouter if YieldForwarder is the keeper. reportAndForward redeems the entire share balance atomically in the same transaction: <pre>uint256 shares = IERC20(strategy).balanceOf(address(this)); if (shares == 0) return 0; assets = IRedeemable(strategy).redeem(shares, receiver, address(this), maxLoss);</pre> After reportAndForward completes, balances[dragonRouter] = 0. The next time a loss occurs and _handleDragonLossProtection runs, Math.min(sharesToBurn, 0) = 0 - no shares are burned, no loss is absorbed. Deploying a strategy with enableBurning = true and YieldForwarder as keeper silently neutralizes the entire loss protection feature.
Recommendations	Document that enableBurning is incompatible with YieldForwarder as keeper.
Comments / Resolution	Acknowledged.

Issue_67	When YieldForwarder is the keeper, sweepAirdrop can only be triggered by management.
Severity	Informational
Description	sweepAirdrop is gated by onlyKeepers (keeper or management). YieldForwarder exposes no method to call arbitrary strategy functions, so when it is the keeper, the sweep responsibility falls entirely on management. If management is a multisig or a contract, this requires a dedicated proposal/execution for each airdrop event.
Recommendations	Confirm that management's operational setup supports this before pairing SparkStrategy with YieldForwarder .
Comments / Resolution	Acknowledged.

Issue_68	Switching the keeper to YieldForwarder is implicitly delayed by the 14-day dragonRouter cooldown
Severity	Low
Description	YieldForwarder requires two simultaneous conditions to operate correctly: it must be both the strategy's keeper (to call report) and its dragonRouter (to receive minted profit shares). Changing these roles has asymmetric timelines - setKeeper takes effect immediately, while setDragonRouter enforces a mandatory 14-day cooldown This creates an implicit constraint: the keeper cannot be switched to YieldForwarder until the dragonRouter change is finalized, meaning the effective timeline for migrating to/from YieldForwarder is always at least 14 days, regardless of operational urgency. This constraint is not stated anywhere in the contract or documentation Since YieldForwarder is an immutable contract, any change in external conditions may require modifying the contract. This would introduce additional operational risks associated with delays. For example, if the keeper that calls YieldForwarder becomes malicious, you won't be able to replace it quickly.
Recommendations	Alternatively, you could provide a quick switch to the new dragon router for the keeper replacement scenario. Or you could document this behavior and keep it in mind. However, this would make the keeper replacement operation dependent on switching the dragon router, and it would no longer be instantaneous.
Comments / Resolution	Acknowledged.

SwappingYieldForwarder

`SwappingYieldForwarder` is an extension of `YieldForwarder` that adds a swap step between share redemption and asset forwarding, enabling cross-asset yield flows where the strategy's underlying asset differs from the desired final asset. It is used within the Octant system when yield needs to be converted (e.g., from DAI to USDS or from wstETH to USDS) before delivery to the receiver, integrating with pluggable ISwapper adapters such as `PSMSwapper`, `CurveSwapper`, `UniswapV3SwapperAdapter`, or `UniswapV4SwapperAdapter` for DEX routing.

The contract provides a dual-mode design acting as a built-in circuit breaker: keepers can call either `reportSwapAndForward` (with swap) or the inherited `reportAndForward` (without swap, direct forwarding) depending on swap protocol availability. In the swap path, profit shares are redeemed to this contract rather than the receiver, the underlying assets are transferred to the configured swapper, and the swapper handles the exchange and delivers target assets directly to the receiver. The swapper enforces `minAmountOut` internally for slippage protection. All configuration (receiver, keeper, targetAsset, swapper) is fully immutable.

Privileged Functions

- `reportSwapAndForward`

Core Invariants:

INV 1: Only the designated keeper can trigger `reportSwapAndForward` or `reportAndForward`.

INV 2: Swapped and forwarded assets can only flow to the hardcoded receiver.

INV 3: The swap output must meet the `minAmountOut` requirement or the transaction reverts.

Issue_69	Missing slippage floor on <code>reportSwapAndForward</code>
Severity	Governance
Description	The <code>reportSwapAndForward</code> function in <code>SwappingYieldForwarder</code> forwards <code>minAmountOut</code> from the keeper straight into the swapper swap path without a contract-enforced minimum. Typical swappers only revert when realized output drops under <code>minAmountOut</code>, so a zero floor accepts arbitrarily bad execution. If a compromised keeper runs <code>reportSwapAndForward</code> against the chosen strategy with <code>minAmountOut</code> zero, the swapper will accept any amount of tokens in return, even a single wei. The hardcoded receiver receives almost nothing, while the compromised keeper could have siphoned these funds to themselves by being an LP on the protocol used for swapping.
Recommendations	Consider setting an immutable or configurable minimum output or acknowledging this issue.
Comments / Resolution	Fixed by adding a <code>minSlippageBps</code> parameter. The governance is still responsible for setting this correctly.

Issue_70	Swap path lacks a caller-chosen deadline
Severity	Low
Description	The swap function carries token addresses, size, <code>minAmountOut</code>, and receiver, but no expiry for the trade intent. <code>UniswapV3SwapperAdapter</code> passes the current <code>block.timestamp</code> into the Uniswap V3 router as the deadline field, which always passes at inclusion and therefore adds no real time bound between quoting and settlement. Market prices can move while the same relative <code>minAmountOut</code> slack stays valid. After the reference price rises, an unchanged percent band below spot implies larger absolute notional leakage to counterparties than when the quote was built. Other <code>ISwapper</code> adapters omit an expiring guard entirely, so the stale-slippage pattern generalizes across swap integrations unless each router enforces its own meaningful deadline.
Recommendations	Consider adding a <code>deadline</code> parameter for <code>reportSwapAndForward</code> and compare that value to the current <code>block.timestamp</code>. This will ensure that calls will not swap with a stale slippage regardless of the swapper they are interacting with.
Comments / Resolution	Fixed following recommendation.

Issue_71	Swaps can revert due to 0 <code>amountSpecified</code>
Severity	Low
Description	Even though the input amount is checked to be non-zero, it is not guaranteed that the output amount will be greater than zero. In such a case, the swap and hence the <code>reportSwapAndForward</code> function can revert. For eg: <code>UniswapV3SwapperAdapter</code> with multi-hop swap. In case the intermediate amount is 0, the swap will revert
Recommendations	Consider acknowledging the issue.
Comments / Resolution	Acknowledged.

CurveSwapper

CurveSwapper is an immutable **ISwapper** adapter for direct Curve **StableSwap** pool exchanges, performing single-pool swaps between correlated or pegged **ERC-20** assets using Curve's **StableSwap** invariant for near-zero slippage. Each deployed instance is configured for a specific pool and token pair via immutables, using direct pool calls rather than the Curve Router for gas efficiency since the exact pool is known at deploy time. It is used by **SwappingYieldForwarder** as a pluggable swap adapter.

The contract validates at construction that the configured tokens match the pool's coins at the specified indices, preventing misconfiguration. During swaps, it uses balance measurement before and after the exchange call for compatibility with legacy Curve pools like 3Pool whose **exchange** does not return output amounts. The contract is fully stateless, holds no tokens between calls, and supports only ERC-20 to ERC-20 swaps. All configuration (pool, indices, tokens) is immutable.

Core Invariants:

INV 1: Minimum output amount (`minAmountOut`) is enforced by the Curve pool's exchange function.

No issues were identified in this contract.

PSMWrapper

PSMWrapper is an immutable **ISwapper** adapter for the Sky Protocol (formerly MakerDAO) Peg Stability Module and **DaiUsds** converter, enabling stablecoin conversions between DAI, USDS, and gem tokens like USDC on Ethereum mainnet. It supports four distinct swap routes - **SELL_GEM** [gem to DAI/USDS], **BUY_GEM** [DAI/USDS to gem], **DAI_TO_USDS**, and **USDS_TO_DAI** - with each deployed instance configured for exactly one route. It is used by **SwappingYieldForwarder** as a pluggable swap adapter.

The contract is fully stateless and holds no tokens between calls. The caller transfers input tokens to the contract before calling **swap**. For PSM routes, the contract handles fee calculations via the protocol's tin/tout parameters and gem amount conversions using the configured **conversionFactor**. For **DaiUsds** converter routes, it enforces strict 1:1 conversion guarantees and reverts if the actual output does not match the input exactly. All configuration [protocol, route, tokens, **conversionFactor**] is set immutably at construction and the swap function validates that input/output tokens match the configured pair.

Core Invariants:

INV 1: **DAI_TO_USDS** and **USDS_TO_DAI** routes must produce exact 1:1 output or the transaction reverts.

INV 2: Swap output must meet the **minAmountOut** requirement.

Issue_72	PSM <code>buyGem</code> strands residual <code>tokenIn</code> dust
Severity	Low
Description	PSMSwapper <code>_buyGem</code> computes <code>gemAmt</code> with floor division on <code>amountIn</code> after fee scaling, then calls <code>buyGem</code> with that integer gem amount. The PSM charges DAI or USDS implied by <code>gemAmt</code> scaled back to eighteen decimals, which is strictly below <code>amountIn</code> whenever the division discarded a fractional gem remainder. The code approves the full <code>amountIn</code> but the protocol pulls only the lower charge, clears approval to zero, forwards <code>tokenOut</code> to receiver, and never returns the leftover stablecoin. <code>ISwapper</code> documentation expects adapters to stay empty between calls; if they are not, the value can be captured by an arbitrary user. Residual <code>tokenIn</code> therefore piles inside <code>PSMSwapper</code> where <code>SwappingYieldForwarder</code> cannot reclaim it through normal report flows, leaving dust that was meant to swap in full available for any actor to take.
Recommendations	After <code>buyGem</code> on the <code>BUY_GEM</code> route, transfer the remaining <code>tokenIn</code> balance to <code>msg.sender</code> so the forwarder receives dust for reuse on later reports.
Comments / Resolution	Fixed by only pulling in the required number of tokens.

Issue_73	PSMSwapper BUY_GEM and SELL_GEM routes revert when LitePSM lacks DAI liquidity
Severity	Low
Description	SwappingYieldForwarder.reportSwapAndForward calls PSMSwapper.swap after redeeming profit shares. For SELL_GEM [USDC -> DAI/USDS] and BUY_GEM [DAI/USDS -> USDC] routes, the swap delegates directly to LitePSM [USDC/DAI] / LitePSMWrapper [USDC/USDS]. LitePSM holds a pre-minted DAI buffer that is depleted by sellGem calls and replenished via fill. If the buffer is empty, either because no fill has been called since the last depletion or because another transaction consumed the remaining DAI in the same block, sellGem reverts, and the entire reportSwapAndForward call reverts with it. fill is permissionless and typically called by MakerDAO keeper bots, but it is not atomic with the swap. There is no guarantee the PSM is liquid at the exact moment the keeper triggers reportSwapAndForward. The critical consequence is that reportSwapAndForward bundles strategy.report, and the swap into a single call. When the PSM has low liquidity, report wait to execute, totalAssets is not updated, profit shares are not minted, and the strategy's accounting falls behind. The swap itself is not time-sensitive, and the yield can be forwarded later; the report call is what matters for protocol health.
Recommendations	If there isn't enough liquidity to execute the report and swap, this needs to be addressed by implementing a fallback mechanism for a deferred swap.
Comments / Resolution	Acknowledged.

UniswapV3SwapperAdapter

UniswapV3SwapperAdapter is an immutable **ISwapper** adapter for Uniswap V3 exact-input swaps, supporting both single-hop and multi-hop (via intermediate base token) routing through the Uniswap V3 **SwapRouter**. Each deployed instance is configured with fixed routing parameters (router, fee tiers, base token) at construction, and the routing mode is automatically determined based on whether the swap tokens match the configured base token. It is used by **SwappingYieldForwarder** as a pluggable swap adapter.

The contract routes swaps through the Uniswap V3 **SwapRouter** using either **exactInputSingle** for single-hop or **exactInput** with path encoding for multi-hop calls. When neither input nor output token is the base, a two-hop path is constructed (tokenIn -> base -> tokenOut) with separate fee tiers for each hop. When a token IS the base, it falls back to single-hop with the appropriate fee tier. Input token approval is set on the router before each swap. The contract is fully stateless, immutable, and holds no tokens between calls.

Core Invariants:

INV 1: Minimum output amount (**minAmountOut**) is enforced by the Uniswap V3 **SwapRouter**.

Issue_74	Immutable Uniswap routes miss deep pools
Severity	Low
Description	UniswapV3SwapperAdapter sets an immutable fee tier at deployment. Uniswap hosts multiple pools for the same token pair, distinguished in part by fee, with different liquidity depth and price impact. Every swap uses only those baked-in pool keys, so execution can run through a thinner pool while deeper same-asset pools exist, reducing realized amountOut for identical input size.
Recommendations	Make fee tiers governable or supplied per swap, or run off-chain monitoring and redeploy when a deeper same-asset V3 pool key becomes dominant.
Comments / Resolution	Acknowledged. Management can re-set the swapper address in the Forwarder, so new swappers will be deployed if better liquidity routes are found.

Issue_75	Unused tokens left stranded in the swapper contract
Severity	Low
Description	In <code>uniswapV3</code>, if the target pool runs out of liquidity, the swap transaction does not revert; instead, it just stops the swap and leaves some of the input tokens unused. The Uniswap router is aware of this behaviour and only charges the amount of tokens from the caller that will actually be swapped. Here, however, the <code>reportSwapAndForward</code> function transfers all <code>amountIn</code> tokens to the swapper already. If the target pool runs out of liquidity, some input tokens will stay with the swapper contract unused. These stranded tokens can then be claimed by anyone who calls the swap function here, since there are no caller guards. Thus, if the target pool runs out of liquidity, some input tokens are stranded, which can then be stolen. Liquidity providers can even frontrun these swap transactions to pull liquidity from Uniswap pools to trigger this condition, although the slippage protection mitigates this issue to some extent.
Recommendations	Consider returning any unused <code>tokenIn</code> to the caller at the end of the swap function.
Comments / Resolution	Fixed by refunding unused tokens.

Issue_76	Approvals not zeroed out in the end
Severity	Informational
Description	In the swap function, the contract grants approval to the router for <code>amountIn</code> amount. <pre><i>IERC20(tokenIn).forceApprove(router, amountIn);</i></pre> However, this approval is not zeroed out at the end of the swap function.
Recommendations	Zero out the approval at the end of the swap function to remove any hanging approvals.
Comments / Resolution	Fixed following recommendations.

UniswapV4SwapperAdapter

`UniswapV4SwapperAdapter` is an immutable `ISwapper` adapter for Uniswap V4 exact-input swaps via the singleton `PoolManager`, supporting both single-hop and multi-hop (via intermediate base token) swap routing. Each deployed instance is configured with fixed pool parameters (fee, tickSpacing, hooks) for deterministic routing, and the routing mode is automatically determined by the base token parameter: `address[0]` for single-hop, or a non-zero address for multi-hop unless the input or output token IS the base token. It is used by `SwappingYieldForwarder` as a pluggable swap adapter.

The contract uses V4's unlock callback pattern; the swap function calls `poolManager.unlock`, which triggers `unlockCallback` back on this contract to execute the actual swap logic and settlement. Settlement uses V4's `sync/transfer/settle` pattern for input tokens and `take` for output tokens, requiring no ERC-20 approvals to the `PoolManager`. For multi-hop swaps, base token deltas net to zero within the `PoolManager's` internal accounting. The callback is gated to only accept calls from the `PoolManager` address, and all configuration is fully immutable.

Privileged Functions

- `unlockCallback`

Core Invariants:

INV 1: Only the `PoolManager` can invoke the `unlockCallback` function.

INV 2: Swap output must meet the `minAmountOut` requirement or the transaction reverts.

INV 3: Input and output token addresses cannot be the zero address.

Issue_77	V4 <code>tickSpacing</code> fixed at deploy for both hops
Severity	Low
Description	<code>UniswapV4SwapperAdapter</code> stores <code>tickSpacing</code> for the first hop and <code>tickSpacingOut</code> for the second hop as immutables chosen only in the constructor. V4 pool keys combine fee, tick spacing, and hooks, so the same two currencies exist across many pools with different spacing and liquidity. Runtime swaps cannot pivot to a better pool without deploying a new adapter. Depth can migrate to another same-asset pool key while this contract keeps using the original <code>tickSpacing</code> and <code>tickSpacingOut</code> , depressing output quality versus the best available pool.
Recommendations	Make <code>tickSpacings</code> and fee tiers governable or supplied per swap, or run off-chain monitoring and redeploy when a deeper same-asset V4 pool key becomes dominant.
Comments / Resolution	Acknowledged. Management can re-set the swapper address in the Forwarder, so new swappers will be deployed if better liquidity routes are found.

Issue_78	Unused tokens are left stranded in the Singleton contract
Severity	Low
Description	In <code>uniswapV4</code>, if the target pool runs out of liquidity, the swap transaction does not revert; instead, it just stops the swap and leaves some of the input tokens unused. The singleton pool contract only charges tokens that are necessary for the swap, not the entire <code>amountIn</code>. Thus, unused tokens can be claimed back via a <code>take</code> call. Here, however, the swapper sends forward all tokens to the singleton contract. It only calls <code>take</code> on <code>tokenOut</code>, leaving any unused <code>tokenIn</code> in the contract. So any unused <code>tokenIn</code> is effectively donated to the singleton contract.
Recommendations	Call <code>take</code> with <code>tokenIn</code> as well to collect any unused input tokens and send them to the <code>msg.sender</code> at the end of the swap.
Comments / Resolution	Fixed following recommendations.